INDEX

NAME: Nithesh G STD.: _____ SEC.: _____ ROLL NO.: 240701365 SUB.: Computer Networks

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.		Basic Networking Commands in Linux And Windows Operating Systems	21/7/26	27/7/26
2.		Learing and Assigning of IP Address manually to Computers		27/7/26
3.		Study of different types of Network Cables And Crimping of cable with RJ45 Connector		27/7/26
4		Analyse the Behaviour of Network devices using CISCO PACKET TRACER Simulation		11/8/26
5.		Design a simple topology And configure with one router, two switches And PCs Using CISCO PACKET TRACER		11/8/26
6.		Setting Up a local Area Network (LAN)		17/8/26
7.		Customizing Switch with Network Module using CISCO PACKET TRACER		18/8/26
8.		Develop a Customized Ping command to test the server connectivity		26

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
9.		Building an Anonymous FTP Scanner Using Python		
10.		Implement Packet sniffing using Raw sockets in Python		
11.		Nmap Live Host Discovery		
12.		Develop a Single Calculator Using XMLRPC		
13.		To capture, save and analyze traffic on TCP/UDP/ARP/ IP/HTTP/DCHP/ICMP/ DNS Using Wireshark tool		
14		Develop a Program to Create Reverse TCP Connection Using Python Sockets		

Experiment - 1

Basic Networking Commands in Linux and Windows operating systems

AIM: Study of various Network Commands used in Linux and Windows

Windows Commands:

1) arp -a

Description:

* Command used to show IP address along with MAC address

2) hostname

Description:

* Command used to display the name of computer visible to the network

3) ipconfig

Description:

* Command used to display the computer's IP network configuration

4) netstat -r

Description:

* Command used to display the computer's routing table
* It contains information both about IPv4 and IPv6

5) ping <domain>

Description:

* used to check if a domain or host is reachable over the network

6) nslookup <domain>

Description:

* Command used to find IP address and DNS information of a domain

7) route print

Description

* Command used to display the computer's routing table

8) getmac

Description:

* Command used to display the MAC address of the computer network adapters

9) Tracert <domain>

Description:

* Command used to trace path taken by data packets to reach a domain or host

Linux Commands

1) ip addr

Description:

* Command used to display the IP addresses and network interface information

2) ip link

Description:

* Command used to display and manage network interfaces

3) hostname

Description:

* Command used to display the system's hostname

4) ping <domain>

Description:

* Command used to check if a host or domain is reachable over the network

5) ip route

Description:

* Command used to display the system's routing table

6) ip neigh

Description:

* Command used to display the IP-to-MAC address mappings of neighboring devices

7) nslookup <domain>

Description:

* Command used to find the IP address and DNS of a domain

Windows Command Outputs:

```
C:\Users\REC>arp -a

Interface: 172.16.9.7 --- 0xb
  Internet Address      Physical Address      Type
  172.16.8.1            7c-5a-1c-cf-be-45     dynamic
  172.16.8.128          00-27-0e-13-eb-34     dynamic
  172.16.8.148          00-27-0e-13-ec-c5     dynamic
  172.16.8.153          00-27-0e-13-ea-3f     dynamic
  172.16.8.193          7c-57-58-39-53-a6     dynamic
  172.16.9.17           00-27-0e-13-ee-c4     dynamic
  172.16.9.35           00-27-0e-13-f4-bd     dynamic
  172.16.9.205          d8-bb-c1-c5-cc-21     dynamic
  172.16.10.14          d8-bb-c1-c5-ca-4b     dynamic
  172.16.10.15          d8-bb-c1-c5-ca-15     dynamic
  172.16.10.63          d8-bb-c1-c5-ca-0c     dynamic
  172.16.10.75          d8-bb-c1-c5-cd-64     dynamic
  172.16.10.92          d8-bb-c1-c5-cb-79     dynamic
  172.16.10.102         d8-bb-c1-c5-cb-74     dynamic
  172.16.10.116         d8-bb-c1-c5-c8-96     dynamic
  172.16.11.255         ff-ff-ff-ff-ff-ff     static
  224.0.0.2             01-00-5e-00-00-02     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static

C:\Users\REC>hostname
DESKTOP-SAR30EO

C:\Users\REC>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::879b:ce53:c106:2a66%11
   IPv4 Address. . . . . . . . . . . : 172.16.9.7
   Subnet Mask . . . . . . . . . . . : 255.255.252.0
   Default Gateway . . . . . . . . . : 172.16.8.1
```

```
C:\Users\REC>netstat -r
===========================================================================
Interface List
 11...00 27 0e 13 ed 89 ......Realtek PCIe GbE Family Controller
  1...........................Software Loopback Interface 1
===========================================================================

IPv4 Route Table
===========================================================================
Active Routes:
Network Destination        Netmask          Gateway       Interface  Metric
          0.0.0.0          0.0.0.0      172.16.8.1      172.16.9.7    281
        127.0.0.0        255.0.0.0         On-link         127.0.0.1    331
        127.0.0.1  255.255.255.255         On-link         127.0.0.1    331
  127.255.255.255  255.255.255.255         On-link         127.0.0.1    331
       172.16.8.0    255.255.252.0         On-link      172.16.9.7    281
       172.16.9.7  255.255.255.255         On-link      172.16.9.7    281
    172.16.11.255  255.255.255.255         On-link      172.16.9.7    281
        224.0.0.0        240.0.0.0         On-link         127.0.0.1    331
        224.0.0.0        240.0.0.0         On-link      172.16.9.7    281
  255.255.255.255  255.255.255.255         On-link         127.0.0.1    331
  255.255.255.255  255.255.255.255         On-link      172.16.9.7    281
===========================================================================
Persistent Routes:
  Network Address          Netmask  Gateway Address  Metric
          0.0.0.0          0.0.0.0      172.16.8.1  Default
===========================================================================

IPv6 Route Table
===========================================================================
Active Routes:
 If Metric Network Destination      Gateway
  1    331 ::1/128                  On-link
 11    281 fe80::/64                On-link
 11    281 fe80::879b:ce53:c106:2a66/128
                                    On-link
  1    331 ff00::/8                 On-link
 11    281 ff00::/8                 On-link
===========================================================================
Persistent Routes:
  None
```

```
C:\Users\REC>nslookup www.google.com
Server:  UnKnown
Address:  172.16.8.1

Non-authoritative answer:
Name:    www.google.com
Addresses:  2001:4860:482d:7700::
          2001:4860:482a:7700::
          2001:4860:482c:7700::
          2001:4860:4826:7700::
          2001:4860:4828:7700::
          2001:4860:4827:7700::
          2001:4860:482b:7700::
          2001:4860:4829:7700::
          142.251.151.119
          142.251.150.119
          142.251.157.119
          142.251.155.119
          142.251.156.119
          142.251.154.119
          142.251.152.119
          142.251.153.119


C:\Users\REC>ping www.google.com

Pinging www.google.com [142.251.151.119] with 32 bytes of data:
Reply from 142.251.151.119: bytes=32 time=3ms TTL=119
Reply from 142.251.151.119: bytes=32 time=3ms TTL=119
Reply from 142.251.151.119: bytes=32 time=3ms TTL=119
Reply from 142.251.151.119: bytes=32 time=3ms TTL=119

Ping statistics for 142.251.151.119:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 3ms, Maximum = 3ms, Average = 3ms
```

```
C:\Users\REC>route print
===========================================================================
Interface List
 11...00 27 0e 13 ed 89 ......Realtek PCIe GbE Family Controller
  1...........................Software Loopback Interface 1
===========================================================================

IPv4 Route Table
===========================================================================
Active Routes:
Network Destination        Netmask          Gateway       Interface  Metric
          0.0.0.0          0.0.0.0      172.16.8.1      172.16.9.7     281
        127.0.0.0        255.0.0.0         On-link         127.0.0.1     331
        127.0.0.1  255.255.255.255         On-link         127.0.0.1     331
  127.255.255.255  255.255.255.255         On-link         127.0.0.1     331
       172.16.8.0    255.255.252.0         On-link      172.16.9.7     281
       172.16.9.7  255.255.255.255         On-link      172.16.9.7     281
    172.16.11.255  255.255.255.255         On-link      172.16.9.7     281
        224.0.0.0        240.0.0.0         On-link         127.0.0.1     331
        224.0.0.0        240.0.0.0         On-link      172.16.9.7     281
  255.255.255.255  255.255.255.255         On-link         127.0.0.1     331
  255.255.255.255  255.255.255.255         On-link      172.16.9.7     281
===========================================================================
Persistent Routes:
  Network Address          Netmask  Gateway Address  Metric
          0.0.0.0          0.0.0.0      172.16.8.1  Default
===========================================================================

IPv6 Route Table
===========================================================================
Active Routes:
 If Metric Network Destination      Gateway
  1    331 ::1/128                  On-link
 11    281 fe80::/64                On-link
 11    281 fe80::879b:ce53:c106:2a66/128
                                    On-link
  1    331 ff00::/8                 On-link
 11    281 ff00::/8                 On-link
===========================================================================
Persistent Routes:
  None
```

```
C:\Users\REC>getmac

Physical Address    Transport Name
=================== ==========================================================
00-27-0E-13-ED-89   \Device\Tcpip_{8DBEED08-BE06-4C98-A68D-13B9C4CEBF30}

C:\Users\REC>tracert www.google.com

Tracing route to www.google.com [142.251.150.119]
over a maximum of 30 hops:

  1    <1 ms    <1 ms    <1 ms  172.16.8.1
  2     1 ms     1 ms     1 ms  137.97.163.129
  3     *        *        *     Request timed out.
  4     4 ms     3 ms     3 ms  172.31.5.65
  5     3 ms     3 ms     4 ms  142.251.150.119

Trace complete.
```

Linux Command Output:

```
root@root ~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:12:6c:4d brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 85704sec preferred_lft 85704sec
    inet6 fe80::d266:8c9d:81ed:6afe/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
root@root ~$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 08:00:27:12:6c:4d brd ff:ff:ff:ff:ff:ff
root@root ~$ hostname
boss
root@root ~$ ping www.google.com
PING www.google.com (142.251.157.119) 56(84) bytes of data.
64 bytes from 142.251.157.119 (142.251.157.119): icmp_seq=1 ttl=63 time=6.35 ms
64 bytes from 142.251.157.119 (142.251.157.119): icmp_seq=2 ttl=63 time=28.0 ms
64 bytes from 142.251.157.119 (142.251.157.119): icmp_seq=3 ttl=63 time=22.6 ms
64 bytes from 142.251.157.119 (142.251.157.119): icmp_seq=4 ttl=63 time=24.3 ms
^C
--- www.google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 6.353/20.305/27.988/8.289 ms
```

```
root@root ~$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
root@root ~$ ip neigh
10.0.2.2 dev enp0s3 lladdr 52:54:00:12:35:02 REACHABLE
10.0.2.3 dev enp0s3 lladdr 52:54:00:12:35:03 REACHABLE
root@root ~$ nslookup www.google.com
Server:         10.0.2.3
Address:        10.0.2.3#53

Non-authoritative answer:
Name:   www.google.com
Address: 142.251.157.119
Name:   www.google.com
Address: 142.251.151.119
Name:   www.google.com
Address: 142.251.154.119
Name:   www.google.com
Address: 142.251.150.119
Name:   www.google.com
Address: 142.251.153.119
Name:   www.google.com
Address: 142.251.156.119
Name:   www.google.com
Address: 142.251.155.119
Name:   www.google.com
Address: 142.251.152.119
Name:   www.google.com
Address: 2001:4860:482b:7700::
Name:   www.google.com
Address: 2001:4860:482c:7700::
Name:   www.google.com
Address: 2001:4860:482d:7700::
Name:   www.google.com
Address: 2001:4860:4827:7700::
Name:   www.google.com
Address: 2001:4860:4828:7700::
Name:   www.google.com
Address: 2001:4860:482a:7700::
Name:   www.google.com
Address: 2001:4860:4826:7700::
Name:   www.google.com
Address: 2001:4860:4829:7700::
```

RESULT:

Thus the study of various Network commands in Linux & Windows is completed and result is obtained

Experiment -2

Learning and Assignment of IP Address manually to Computers

AIM:

To manually assign a IP to the computer

PROCEDURE:

1) Open Control Panel
2) Go to "Network and Internet" -> "Network and Sharing Center"
3) Click Change adapter settings
4) Right click the active network adapter
5) Click on "Properties"
6) Select "Internet Protocol Version 4"
7) Click on "Properties"
8) Select "Use the following IP address"
9) Enter the IP address, Subnet mask, Default gateway
10) Click "Ok" then "Close"

RESULT:

Thus a IP is manually assigned to a computer

Experiment - 3

Study of different types of Network Cables and Crimping of cable with RJ45 Connector

AIM:

To Study different types of Network Cables and Crimping of cable with RJ45 Connector

Network Cables:

1) UTP (Unshielded Twisted Pair):

* UTP cable has twisted copper wire pairs without shielding.
* It is low cost, easy to install, flexible and lightweight
* But it is affected by EMI and has less protection from noise

2) STP (Shielded Twisted Pair):

* STP cable has twisted copper wire pairs without shielding
* It has better protection against EMI, more reliable signal, and improved security

3) Coaxial Cable:

* Coaxial Cable contain a central copper conductor insulation, metallic shield, and outer cover.
* Used in cable TV, CCTV, broadband, etc
* It has brood resistance to interference, strong and can carry signals over longer distances
* But it is bulkier, less flexible, and not commonly used in modern LAN networks

4) Fiber Optic Cable:

* Fiber cable transmits data using pules of light through glass or plastic fibers
* Used for high-speed internet and long-distance communication
* It has very high speed, long transmission distance, low signal loss and immune to EMI
* But it is more expensive, delicate, and requires special tools and skills for installation and repair

Crimping:

* Crimping is the process of attaching an RJ45 connector to the end of an Ethernet cable using a crimping tool
* First, remove outer insulation and arrange the 8 wires inside the cable according to T568B color standard
* Straighten the wires, cut them evenly, and insert them fully into RJ45 connector
* Place the connector into the ~~RJ~~ crimping tool and press firmly to lock the pins onto the wires
* Finally test cable using cable tester

RJ45 T568B Wire Color Order:

1. White-Orange
2. Orange
3. White-Green
4. Blue
5. White-Blue
6. Green
7. White-Brown
8. Brown

RESULT:

Thus the Study of different types of Network cables and Crimping of Cable with RJ45 connector is completed

Lee 27/12/25

Experiment - 4

Analyse the Behaviour of Network devices using CISCO PACKET TRACER Simulation

AIM:

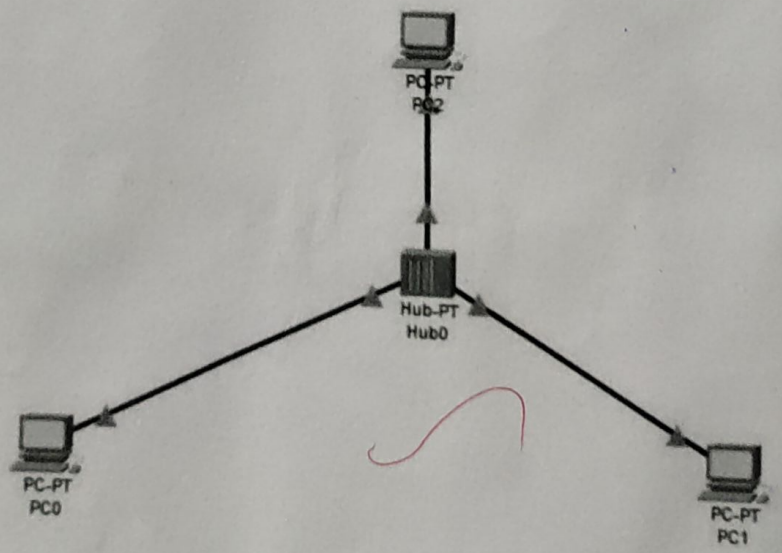
Design a simple network with multiple nodes and use both the (HUB and SWITCH) for a network and find out functioning difference between switch and hub

PROCEDURE:

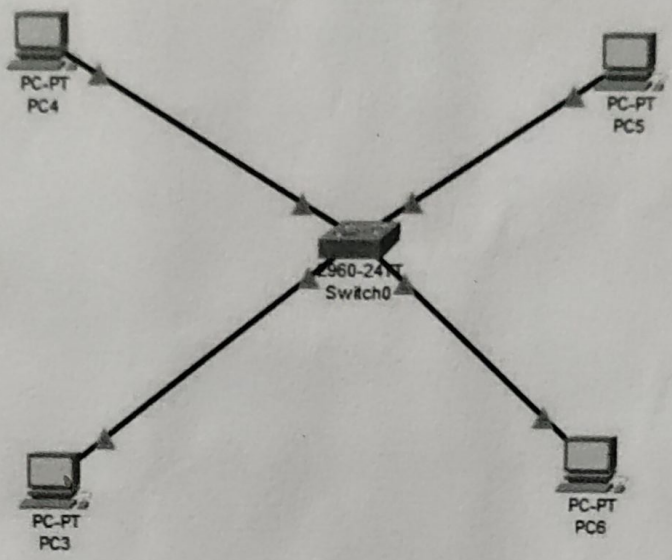
1. Drag and drop 3 Generic PCs and One Hub
2. Drag and drop 4 Generic PCs and One Switch
3. Click on Connections and select Copper Straight-Through Cable
4. Connect the PCs with their respective Hub And Switch
5. Click on PCs, go to Desktop tab, click IP configuration and enter IP address

6. Click on PDU and click on one PC to set as source, then click on another PC to set as destination
7. Observe the flow of PDU from PC to PC
8. Observe how HUB and Switch forward the PDU

Hub Network



Switch Network



RESULT:

Thus a simple network with multiple nodes is constructed using HUB and SWITCH and the functioning differences between them is noted

Experiment-5

Design a simple topology and configure with one router, two switches and PCs using CISCO PACKET TRACER

AIM:

To Design a simple topology and configure with one router, 2 switches and PCs using CISCO PACKET TRACER

PROCEDURE:

I. Build the Topology

1. Add end devices: Add 4 PCs, place PC0 & PC1 on left (LAN1) and PC2 & PC3 on right (LAN2)
2. Place 2 Cisco 2960 switches on left and right
3. Add Cisco 2331 Router in middle
4. Connect the following:
   - PC0, PC1 to Switch0
   - PC2, PC3 to Switch1
   - Switch0, Switch1 to Router0

II. Configure PCs

1. Click on PC
2. Go to Desktop and IP configuration
3. Fill out the following for the PCs

PC0: IP = 192.168.10.10
Gateway = 192.168.10.1

PC1: IP = 192.168.10.11
Gateway = 192.168.10.1

PC2: IP = 192.168.20.10
Gateway = 192.168.20.1

PC3: IP = 192.168.20.11
Gateway = 192.168.20.1

III) Configure Router Interface

1. Click on Router0 and click on CLI tab
2. Press Enter after typing "no" for initial configuration
3. Execute the following commands:

```
Router> enable
Router# configure terminal
Router(config)# interface gigabitEthernet 0/0/0
Router(config-if)# ip address 192.168.10.1 255.255.
Router(config-if)# no shutdown
Router(config-if)# exit
```

```
Router(config)# interface GigabitEthernet 0/0/1
Router(config-if)# ip address 192.168.20.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# end
Router# write memory
```

IV) Verify Connectivity

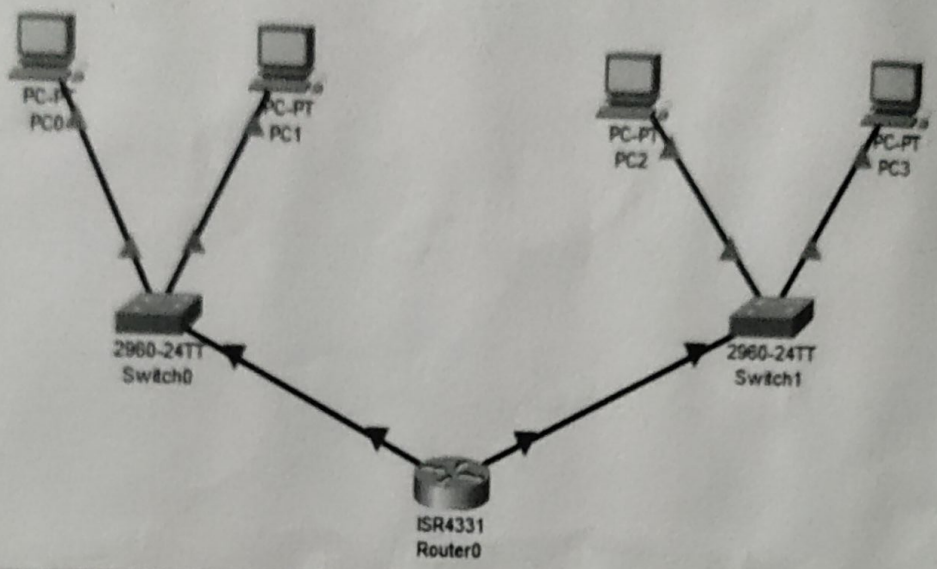
1. Click on PC0, go to the Desktop tab, open the Command prompt

2. Run a command-line ping to your local gateway to confirm local-link

```
ping 192.168.10.1
```

3. Test inter-subnet routing across the network layer by running a cross-network ping to a PC on LAN2

```
ping 192.168.20.10
```

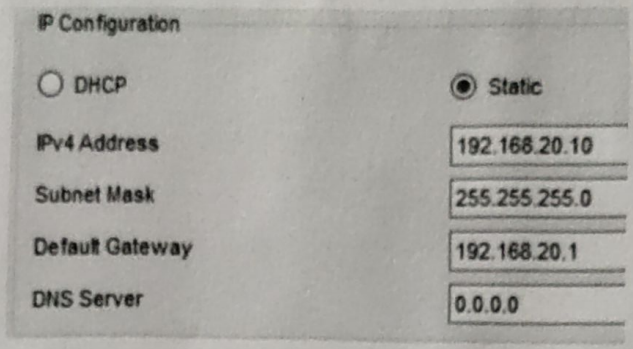


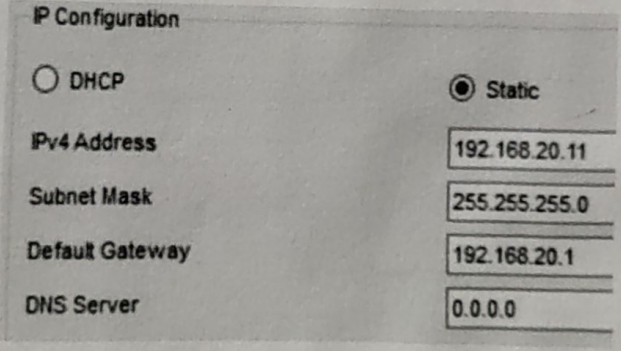
IP Configuration

DHCP	Static
IPv4 Address	192.168.10.10
Subnet Mask	255.255.255.0
Default Gateway	192.168.10.1
DNS Server	0.0.0.0

IP Configuration

DHCP	Static
IPv4 Address	192.168.10.11
Subnet Mask	255.255.255.0
Default Gateway	192.168.10.1
DNS Server	0.0.0.0






```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface GigabitEthernet0/0/0
Router(config-if)#ip address 192.168.10.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to up

Router(config-if)#exit
Router(config)#interface GigabitEthernet0/0/1
Router(config-if)#ip address 192.168.20.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/1, changed state to up

Router(config-if)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#write memory
Building configuration...
[OK]
```

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.10.1

Pinging 192.168.10.1 with 32 bytes of data:

Reply from 192.168.10.1: bytes=32 time=2ms TTL=255
Reply from 192.168.10.1: bytes=32 time<1ms TTL=255
Reply from 192.168.10.1: bytes=32 time<1ms TTL=255
Reply from 192.168.10.1: bytes=32 time<1ms TTL=255

Ping statistics for 192.168.10.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 2ms, Average = 0ms
```

```
C:\>ping 192.168.20.10

Pinging 192.168.20.10 with 32 bytes of data:

Request timed out.
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127
Reply from 192.168.20.10: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.20.10:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

RESULT:

The network topology is operational. The first individual packet of an inter-subnet ping may initially time out due to Address Resolution Protocol (ARP) discovery, but subsequent responses will return Successful. The Router is managing cross-subnet traffic between LAN1 & LAN2

Experiment-6

Setting Up a local Area Network (LAN)

I) AIM

To design, set up and configure LAN connecting 4 computers through network switch, assign static IP, verify connection using ping command and demonstrate resource/ data sharing across network

II) Equipment Required

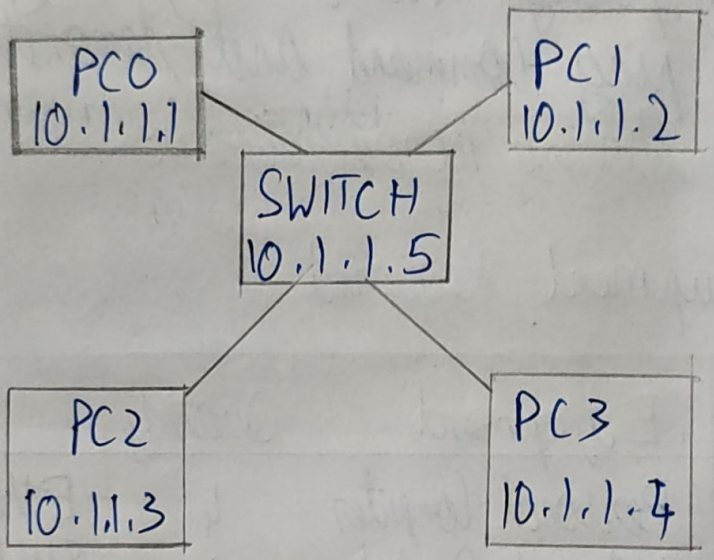
S.No	Equipment	Quantity	Specifications
1	Personal Computer	4	Ethernet (NIC) enabled
2	Network Switch	1	8/16/24 port unmanaged /managed
3	Ethernet Cable	4	Straight-through, R-J45 Connectors

III) Theory:

A LAN is a network that connects computers and devices within a limited geographic area, allowing them to share resources such as files, printers and internet connections

A Star Topology is used in this experiment where all PCs are individually connected to the switch. Switch operates a layer 2 of the OSI model and forwards data frames between devices based on MAC address. Every device in LAN must be assigned an IP address within same network in order to communicate

IV) Network Diagram



V) IP Address Configuration Table

Device	IP. Address	Subnet Mask	Connected To
PC0	10.1.1.1	255.0.0.0	Switch Port Fa0/1
PC1	10.1.1.2	255.0.0.0	Switch Port Fa0/2
PC2	10.1.1.3	255.0.0.0	Switch Port Fa0/3
PC3	10.1.1.4	255.0.0.0	Switch Port Fa0/4
Switch	10.1.1.5	255.0.0.0	VLAN 1 Interface

IV) Implementation in CISCO PACKET TRACER

1) Place the devices:
   - Drag 4 PCs into the workspace
   - Drag 1 switch into the workspace

2) Cable the Devices
   - Using copper Straight-Through cable connect the 4 PCs to the switch

3) Assign IPs
   - Click PC → Desktop → Ip configuration
   - Set IP and Subnet Mask

4) Configure Switch IP
   - Click on switch → CLI tab
   - Enter the commands:

```
enable
configure terminal
interface vlan 1
ip address 10.1.1.5 255.0.0.0
no shutdown
exit
```

5) Verify Connectivity Using Ping

- Click PC → Desktop → Command Prompt
- Use Ping command to other PC : ping 10.1.1.2

6) Demonstrate Data Sharing
- Add server device
- Assign IP address
- open server config and enable HTTP
- From another PC open web Browser and enter IP address to load page

7) Observe Packet Flow in Simulation Mode
- Switch to Simulation Mode
- Send ping or web browser request between 2 devices

RESULT:

All 4 PCs were successfully connected to the switch in star topology and assigned unique IP with same subnet. Ping tests between all devices returned successfull. The shared folder/server resource was successfully accessed from computers on network, confirming that the LAN was functioning correctly for resource sharing

Experiment - 7

Customize Switch with Network Modules Using CISCO PACKET TRACER

AIM:

To Customize Switch with Network Modules Using Cisco Packet Tracer

PROCEDURE:

1) Select and Place Modular Switch
   - Select Switch-PT-Empty and drag it onto the workspace
   - Drag 2 PCs and one Router 2911 onto the workspace

2) Power off the Switch Chassis:
   - Click on Switch
   - Select Physical tab
   - Locate Power Switch and Click it

3) Network Modules
   - Click on PT-SWITCH-NM-1CE and Drag 2 of these modules into Slot 0 & 1
   - Click on PT-SWITCH-NM-1FGE and drag it into Slot 2

4) Power on the Switch and observe Boot sequence
- Click on Power Switch to Turn On
- Switch to CLI
- observe the CISO IOS bootup process

5) Verify Port Recognition via Cisco IOS CLI
- Run the following commands:
  enable
  show ip interface brief
- You should see the newly added interfaces:
  * FastEthernet0/1
  * FastEthernet0/2
  * GigabitEthernet2/1

6) Cabling and Link Testing
- Connect PC0 & PC1 to FastEthernet0/1 and FastEthernet1/1 using a Copper Straight-Through Cable
- Connect GigabitEthernet2/1 to router's fiber interface using Fiber cable
- Configure IP addresses on the PCs and test connectivity using ping

Add a HWIC Fiber Module to Router

1) Power Down 2911 Router
- Click on Router 2911
- Click on Physical tab
- Locate Power switch and click it to turn off

2) Install the HWIC-1GE-SFP Module
- On the left panel, click on HWIC-1GE-SFP
- Drag the HWIC-1GE-SFP card into the empty slot bay on the back of Cisco 2911 Router

3) Power On the Router
- Flip Power Switch back to on
- Switch to CLI tab and wait for it to finish booting

4) Connect the Fiber Cable
- Click on Fiber Cable under Connections Palette
- Click on the Switch then Select GigabitEthernet2/1
- Click on Router 2911 and select newly added GigabitEthernet 0/0/0 port

Router Configuration:

1) Step-by-Step CLI Configuration
- Run the following command sequence

```
enable
configure terminal
interface GigabitEthernet 0/0/0
ip address 192.168.10.1 255.255.255.0
no shutdown
exit
exit
write Memory
```

2) Verification Commands:

i) Check Interface Status & IP Address
- show ip interface brief

Expected Output:

Interface	Protocol	IP-Address	OK?	Method	Status
GigabitEthernet0/0	down	unassigned	YES	unset	administratively down
GigabitEthernet0/1	down	unassigned	YES	unset	administratively down
GigabitEthernet0/2	down	unassigned	YES	unset	administratively down
GigabitEthernet0/0/0	up	192.168.10.1	YES	manual	up

ii) View Hardware Details
- show interfaces GigabitEthernet 0/0/0
- Ensure media type is SFP

3) Connectivity Test
- Run Ping Command from PC0: ping 192.168.10.2

RESULT:
Thus to customize Switch with Network Module is completed using CISCO PACKET TRACER

Experiment-8

Develop a customized ping command to test the server connectivity

AIM:

To develop a customized Ping command in Python that checks the connectivity between a client and a server by sending ICMP Echo Request packets and displaying response time and status

REQUIREMENTS:

* Computer/Laptop
* Internet Connection
* Windows 10/11 or Linux

THEORY:

A ping command is a network diagnostic tool used to determine whether a host is reachable over the network

It works using the ICMP (Internet Control Message Protocol)

PYTHON CODE:

```
import subprocess, platform

print("="*50)
print("Customized Ping Utility")
print("="*50)
host = input("Enter Website or IP Address: ")
sys = platform.system()
if sys == "Windows":
    cmd = ["ping", "-n", "4", host]
else:
    cmd = ["ping", "-c", "4", host]
try:
    res = subprocess.run(cmd, capture_output=True, text=True)
    print("\nPing Results\n")
    print(result.stdout)
    if result.returncode == 0:
        print("Server Status: Reachable")
    else:
        print("Server Status: Unreachable")
except Exception as e:
    print("Error:", e)
```

RESULT:

Thus packet sniffing using raw sockets is implemented in python and executed successfully

OUTPUT:

```
**********************************************
          Customized Ping Utility
**********************************************

Enter Website or IP Address:
google.com

Pinging google.com [142.250.193.78] with 32 bytes

Reply from 142.250.193.78:
bytes=32 time=28ms TTL=116

Reply from 142.250.193.78:
bytes=32 time=30ms TTL=116

Reply from 142.250.193.78:
bytes=32 time=27ms TTL=116

Reply from 142.250.193.78:
bytes=32 time=29ms TTL=116


Ping statistics for 142.250.193.78

Packets:
Sent = 4
Received = 4
Lost = 0 (0% loss)

Approximate round trip times

Minimum = 27ms
Maximum = 30ms
Average = 28ms
```

RESULT:

Thus A customized Ping Command is developed in Python and executed Successfully

Experiment - 9

Building an Anonymous FTP Scanner Using Python

AIM:

To develop Python-based anonymous FTP scanner using built-in ftplib module and determine whether FTP server permits anonymous access

REQUIREMENTS:

* Windows / Linux
* Python 3
* Builtin ftplib module
* FTP Server
* Network

PYTHON CODE:

```
from ftplib import FTP, all_errors

def ftp_scan(host, port=21):
    print("=" *50)
    print(" ANONYMOUS FTP SCANNER")
    print("=" *50);

    print(f"Target: {host}")
    print(f"Port: {port}")
    print()
```

```
ftp = FTP()
try:
    print("[i] Connecting to FTP server...")
    ftp.connect(host, port)
    print("[+] FTP server is reachable")
    print("[+] Server response:", ftp.getwelcome())
    print("\n[2] Testing anonymous login...")
    ftp.login(user="anonymous", passwd="")
    print("[+] Anonymous login SUCCESSFUL")
    print("[!] The FTP server permits anonymous access.")
    print("[3] Checking accessible directory...")
    files = ftp.nlst()
    if files:
        print("[+] Directory listing is available:")
        for item in files:
            print(" ", item)
    else:
        print("[-] Directory is empty or listing is
              unavailable")
    ftp.quit()
except all_errors as error:
    print("[-] Anonymous login failed or FTP
          connection failed")
    print("[!] Error:", error)
ftp.close()
```

target = input ("Enter FTP server address: ")
ftp-scan (target)

OUTPUT: Success

```
Enter FTP server address: 127.0.0.1
==============================================
 ANONYMOUS FTP SCANNER
==============================================
Target: 127.0.0.1
Port : 2121

[1] Connecting to FTP server...
[+] FTP server is reachable
[+] Server response: 220 pyftpdlib 2.2.0 ready.

[2] Testing anonymous login...
[+] Anonymous login SUCCESSFUL
[!] The FTP server permits anonymous access.

[3] Checking accessible directory...
[+] Directory listing is available:
  .Rhistory
  Harini S - Zoho
  KingsoftData
  My Music
  My Pictures
  My Videos
  anaconda_projects
  desktop.ini
  exp8 -CN
  ftp_anon.py
  sniff.py
```

OUTPUT: Failed to Connect

```
Enter FTP server address: 0.0.0.0
==============================================
 ANONYMOUS FTP SCANNER
==============================================
Target: 0.0.0.0
Port : 2121

[1] Connecting to FTP server...
[-] Anonymous login failed or FTP connection failed.
[!] Error: [WinError 10049] The requested address is not valid in its context
```

RESULT:

Thus the Python based Anonymous FTP server is created and executed

Experiment - 10

Implement packet Sniffing using Raw Sockets in Python

AIM:

To implement packet sniffing using raw sockets in Python on Windows

REQUIREMENTS:

* Windows
* Python 3
* Builtin socket module
* Network

PYTHON CODE:

```
import socket
import struct

sniffer = socket.socket(socket.AF_INET, socket.SOCK_
                        RAW, socket.IPPROTO_IP)

host = socket.gethostbyname(socket.gethostname())
sniffer.bind((host, 0))
sniffer.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

print("Packet Sniffer started...\n")
```

```
try:
    while True:
        raw, addr = sniffer.recvfrom(65535)
        ip_h = raw[0:20]
        iphd = struct.unpack('!BBHHHBBH4s4s', ip_h)

        v = iphd[0] >> 4
        ihl = iphd[0] & 0xF
        ttl = iphd[5]
        protocol = iphd[6]

        src = socket.inet_ntoa(iphd[8])
        dst = socket.inet_ntoa(iphd[9])
        print("-"*20)
        print("Version: ", v)
        print("Header length: ", ihl*4)
        print("TTL: ", ttl)
        print("Protocol: ", protocol)
        print("Source IP: ", src)
        print("Destination: ", dst)
        print("Packet length: ", len(raw))

except KeyboardInterrupt:
    sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)

print("Stopped")
```

OUTPUT

```
Packet Sniffer Started...

--------------------------------------
Version : 4
Header Length: 20
TTL : 1
Protocol : 17
Source IP : 172.16.11.11
Destination : 239.255.255.250
Packet Length: 204
--------------------------------------
Version : 4
Header Length: 20
TTL : 1
Protocol : 17
Source IP : 172.16.8.189
Destination : 239.255.255.250
Packet Length: 652
--------------------------------------
Version : 4
Header Length: 20
TTL : 255
Protocol : 17
Source IP : 172.16.11.108
Destination : 224.0.0.251
Packet Length: 105
--------------------------------------
Version : 4
Header Length: 20
TTL : 1
Protocol : 17
Source IP : 172.16.11.204
Destination : 224.0.0.251
Packet Length: 283
--------------------------------------
```

RESULT:

Thus the packet sniffer is implemented in Python using Raw sockets and it is executed successfully

Experiment-11
Nmap live Host Discovery

AIM:

To discover live hosts on network using Nmap host-discovery scans - ARP scan, ICMP scan and TCP/UDP ping scan and to understand which protocol operates at which layer of the TCP/IP model and when each one applies in TryHackMe

ARP Host Discovery:

Command: $sudo nmap -PR -sn 10.200.6.0/24

Output:

```
Starting Nmap 7.80 ( https://nmap.org )
Nmap scan report for 10.200.6.1
Host is up.
Nmap done: 256 IP addresses (1 host up) scanned in 10.51 seconds
```

ICMP Host Discovery

* ICMP offers several request/reply pairs that can reveal a live host
* The options in nmap for ICMP are (-PE/-PP/-PM)

ICMP Echo (-PE):

Command: $ sudo nmap -PE -sn 10.200.6.0/24

Output:

```
Nmap scan report for 10.200.6.1
Host is up (0.069s latency).
Nmap scan report for 10.200.6.50
Host is up (0.070s latency).
Nmap scan report for 10.200.6.250
Host is up (0.069s latency).
Nmap done: 256 IP addresses (3 hosts up) scanned in 3.38 seconds
```

ICMP Timestamp (-PP):

Command: $ sudo nmap -PP -sn 10.200.6.0/24

output:

```
Nmap scan report for 10.200.6.50
Host is up (0.069s latency).
Nmap scan report for 10.200.6.250
Host is up (0.069s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in 8.44 seconds
```

ICMP Address Mask (-PM)

Command: $nmap -PM -sn 10.200.6.0/24

Output:

```
Nmap done: 256 IP addresses (0 hosts up) scanned in 52.14 seconds
```

TCP and UDP Ping Discovery

* It is used when ICMP is filtered
* TCP & UDP packets can infer that the host is alive merely from the fact that it responds at all

ICP SYN Ping (-PS):

Command: $nmap -PS -sn 10.200.6.0/24

Output

```
Nmap scan report for 10.200.6.50
Host is up (0.070s latency).
Nmap scan report for 10.200.6.250
Host is up (0.068s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in 33.88 seconds
```

TCP ACK Ping (-PA)

Command: $ sudo nmap -PA -sn 10.200.6.0/24

~~Output:~~

Output

```
Nmap scan report for 10.200.6.50
Host is up (0.070s latency).
Nmap scan report for 10.200.6.250
Host is up (0.069s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in 36.64 seconds
```

UDP Ping (-PU)

Command: $ sudo nmap -PU -sn 10.200.6.0/24

Output:

```
Nmap scan report for 10.200.6.50
Host is up (0.070s latency).
Nmap scan report for 10.200.6.250
Host is up (0.069s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in 35.97 seconds
```

RESULT:

Thus ~~N/A~~ Nmap live host discovery is performed using ARP scan, ICMP scan and TCP/UDP ping scan in TryHackMe Platform

Experiment-12
Develop A Simple Calculator Using XMLRPC

AIM:

To design, implement and verify a remote calculator application using the XML-RPC protocol in Python based on client-server architecture

REQUIREMENTS:

* Understanding of client-server architecture and Sockets
* Python
* Fundamentals of HTTP and XML

ALGORITHMS:

1) Server:

i) import from xmlrpc.server
ii) Define the arithmetic functions
iii) Bind the rpc server to localhost port 8000
iv) Register all the functions with rpc
v) Server forever

2) Client:
i) import xmlrpc
ii) Bind to localhost 8000
iii) Accept 2 numbers from user
iv) Invoke remote methods using rpc client
v) Print the outputs returned by server

PYTHON CODE:

1) Server:

```
from xmlrpc.server import SimpleXMLRPCServer
add = lambda a,b: a+b
sub = lambda a,b: a-b
mul = lambda a,b: a*b
dive = lambda a,b: a/b if b!=0 else "Division by zero is not allowed"

server = SimpleXMLRPCServer(("localhost",8000))
print("XML-RPC Calculator Server Running on Port 8000...")
server.register_function(add,"add")
server.register_function(sub,"subtract")
server.register_function(mul,"multiply")
server.register_function(dive,"divide")
server.serve_forever()
```

2) Client

```
import xmlrpc.client
proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
a = float(input("Enter First Number: "))
b = float(input("Enter Second Number: "))

print("\nAddition=", proxy.add(a,b))
print("\nSubtraction=", proxy.subtract(a,b))
```

```
print("Multiplication =", proxy.multiply(a, b))
print("Division =", proxy.divide(a, b))
```

Client Output:

```
Enter First Number: 10
Enter Second Number: 5

Addition = 15.0
Subtraction = 5.0
Multiplication = 50.0
Division = 2.0
```

Server Output:

```
XML-RPC Calculator Server Running on Port 8000...
127.0.0.1 - - [29/Sep/2026 15:31:07] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 15:31:09] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 15:31:11] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [29/Sep/2026 15:31:13] "POST / HTTP/1.1" 200 -
```

RESULT:

The XML-RPC based simple calculator was executed successfully. The procedures for addition, subtraction, multiplication and division were registered on the server and remotely invoked by the client application.

Experiment-13

To Capture, Save, and analyze network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP/DNS Using Wireshark Tool

AIM:

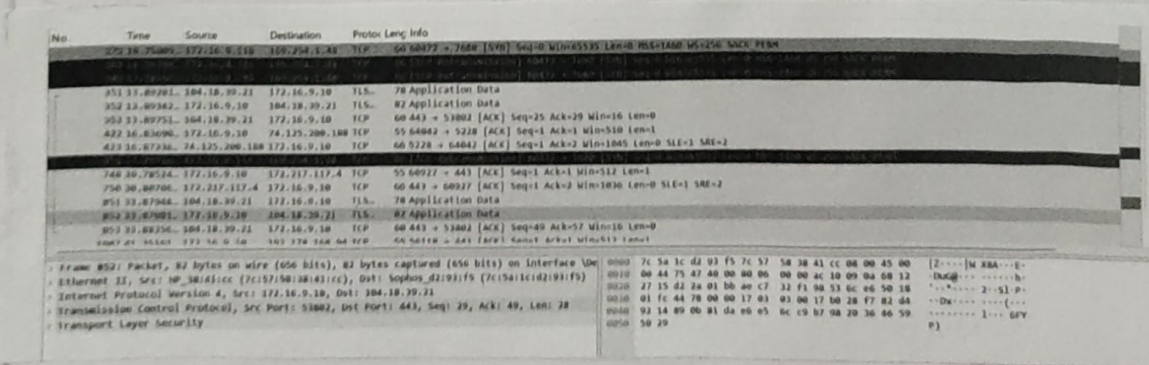
Experiments on Packet capture tool, Wireshark

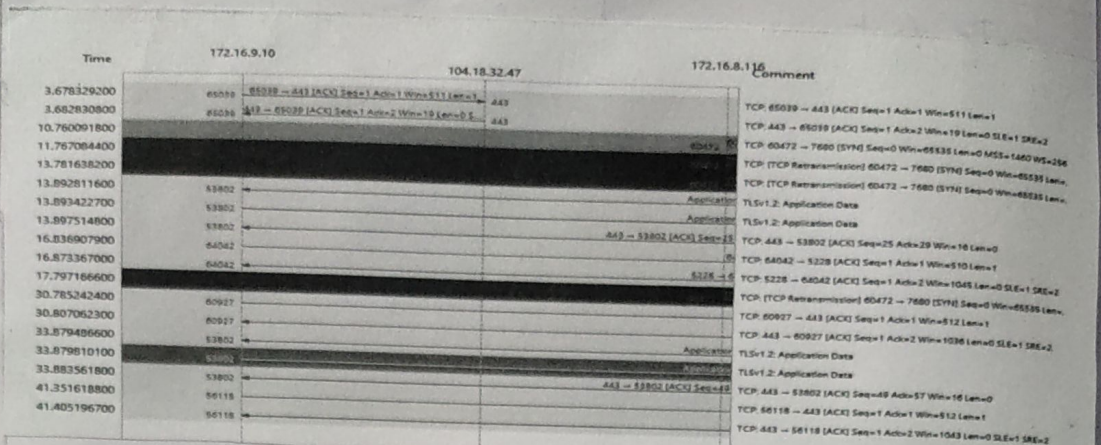
DESCRIPTION:

Wireshark, a network analysis tool, captures packets in real time and displays them in human-readable format. It includes filters, color coding, and other features that help analyze network traffic and inspect packets

I) Capturing TCP/UDP packets

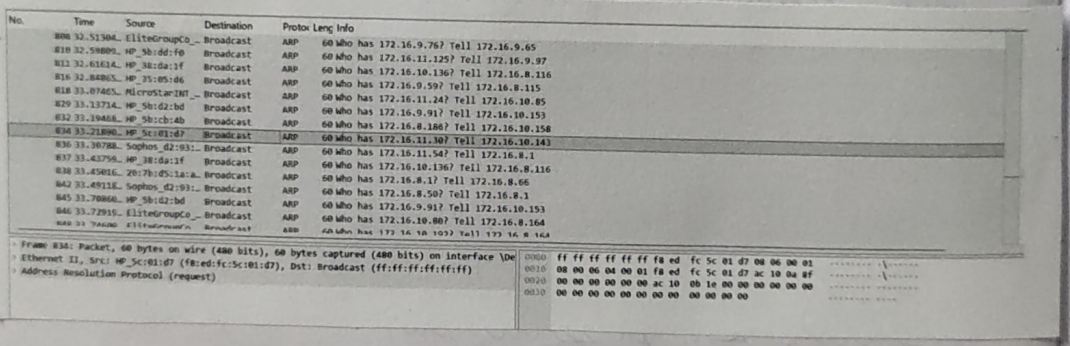
1. Select Start Capture
2. Search TCP in search bar
3. Click Statistics, then Flow graph
4. Save the packets





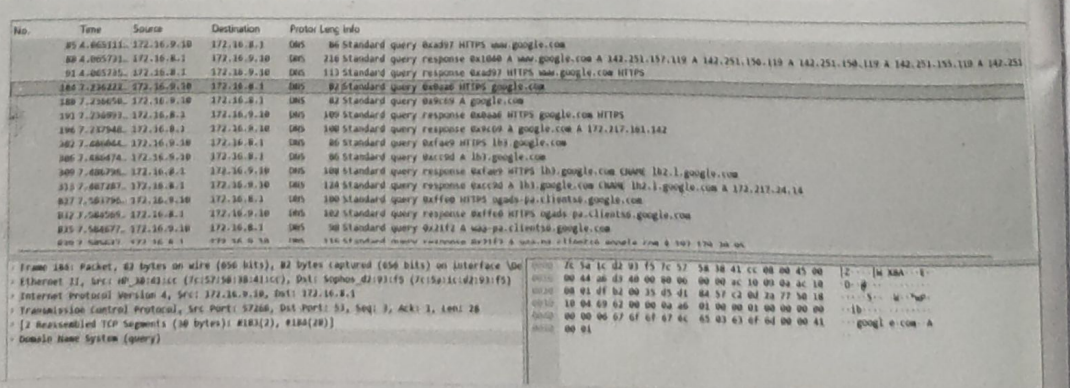
II) Capture ARP packets

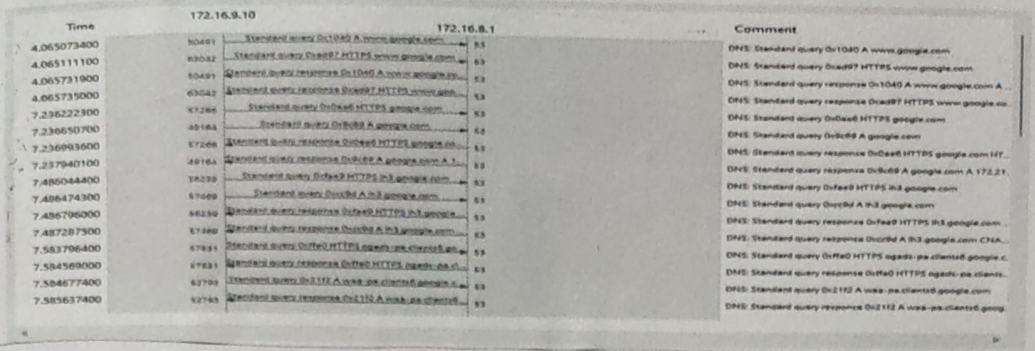
1. Select Capture
2. Search ARP
3. Save packets



III) Capture DNS packets

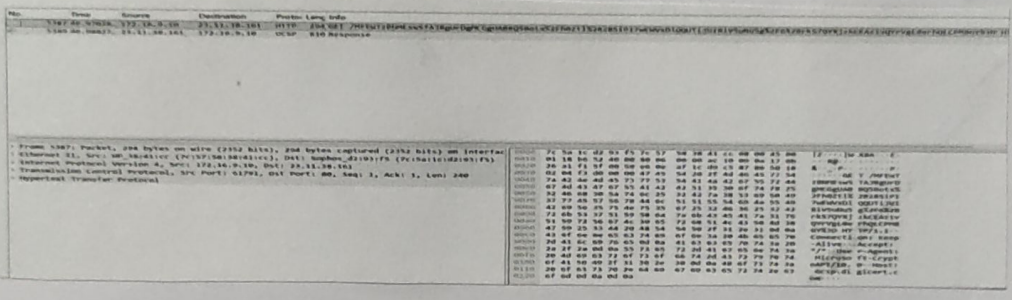
1. Select Capture
2. Search DNS
3. Save packets





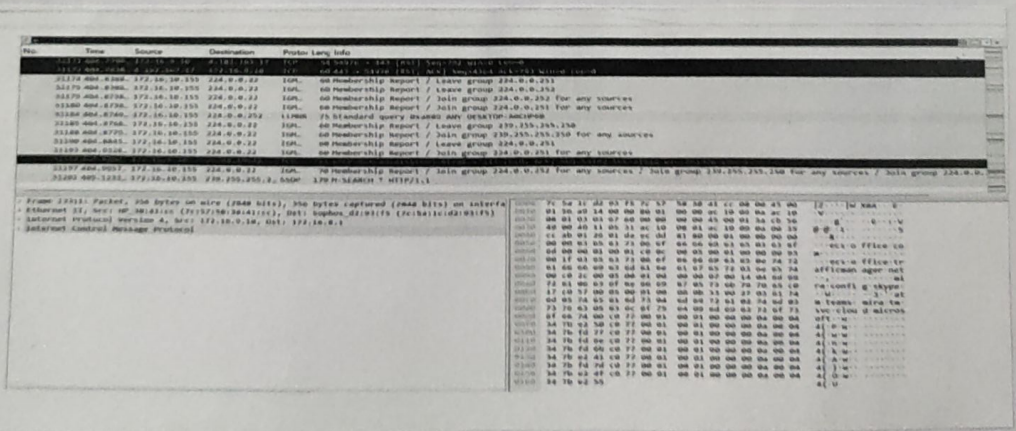
IV) Capture HTTP packets

1. Select Capture
2. Search HTTP
3. Save packets



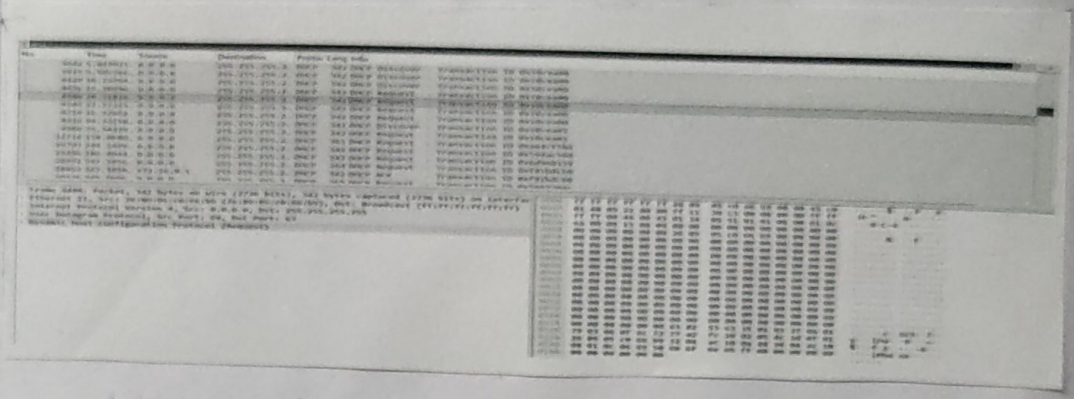
V) Capture IP/ICMP packets

1. Select Capture
2. Search ICMP/IP
3. Save packets



VII) Capture DHCP packets

1. Click Capture
2. Search DHCP
3. Save Packets



RESULT:

Thus the different packet types are captured and analyzed using wireshark.

Experiment -14

Develop a Program to Create a Reverse TCP Connection Using Python Sockets

AIM:

To design, implement, and verify a TCP socket-based reverse connection demonstration in Python using a listener and a connector running on localhost

REQUIREMENTS:

* Basic understanding of client-server architecture and TCP communication
* Python knowledge
* Network

PYTHON CODE:

For listener.py

```
import socket
HOST = "127.0.0.1"
PORT = 5000
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)
print("Waiting for reverse connection...")
con, addr = server.accept()
print("Connected from:", addr)
```

```
msg = con.recv(1024).decode()
print("Received:", msg)
con.send(b"Hello from listener")
con.close()
server.close()
```

connector.py:

```
import socket

HOST = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print("Connecting to listener...")
client.connect((HOST, PORT))
client.send(b"Hello from reverse connector")
rep = client.recv(1024).decode()
print("Listener replied:", rep)
client.close()
```

```
Waiting for reverse connection...
Connected from: ('127.0.0.1', 52144)
Received: Hello from reverse connector
```

```
Connecting to listener...
Listener replied: Hello from listener
```

RESULT:

Thus the python program to create reverse TCP connection using python sockets is completed and executed successfully